

FSD / FWD

Semester 1, 2026

Week 08

Dipto.pratyaksa@rmit.edu.au

**Full stack- *part 2* & Authentication in
React app;**

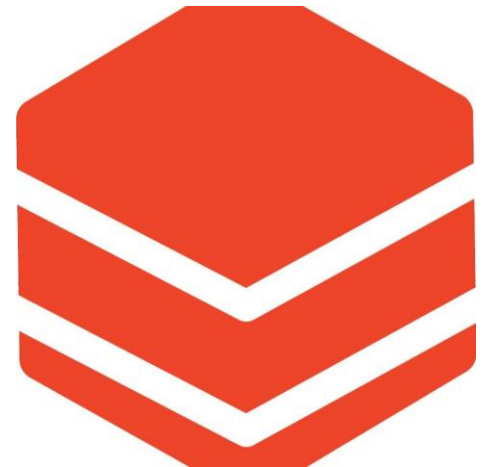
Segment 1

Full stack development – *part 2*

More on database programming

ORM (TypeORM) &

Writing your own API



Database programming

- In week 7's lectional- we covered a big example (*example 1*) of a full stack app –
- In this example
 - We used node, express & TypeORM project that talks to Cloud MS SQL database and creates a table called 'user'. It also performed CRUD operations on this table ie create user, update and delete user. All of these operations were performed by an API written as a part of this project
 - We also created a react app that talked to above API to talk to MS SQL database

Database programming

- In this week's lectorial, we will look at second example
 - example 1 
- Here we will create a table 'tutorial' in the Cloud MS SQL database via node, express and TypeORM project.
- We will also create an API to perform database CRUD operations
- We will then call this API in our second project ie the React front end app
- This will re-instate what you learned in week 7!

example01: node, express and TypeORM project

- We will create the project with `npm init` command
- Update `server.js` file for port number, etc.
- Download requisite node modules
 - `npm install express cors typeorm mysql`
- The actual code can be added in `src/` or `app/` folder of the project directory.
- The directory structure is explained on the next slide.

example01: node, express and TypeORM project

- `data-source.ts`
 - Contains details to connect to Cloud MS SQL database
- Controller file
 - Contains logic of API methods
- Model / Entity file
 - TypeORM syntax to create 'tutorial' table
- `index.ts`
 - Initialise TypeORM, Express
 - Add more database related code
- Routes file
 - Create URL endpoints for API methods

example01: node, express and TypeORM project

- **Methods in the REST API are**

Methods	Urls	Actions
GET	api/tutorials	get all Tutorials
GET	api/tutorials/:id	get Tutorial by id
POST	api/tutorials	add new Tutorial
PUT	api/tutorials/:id	update Tutorial by id
DELETE	api/tutorials/:id	remove Tutorial by id
DELETE	api/tutorials	remove all Tutorials
GET	api/tutorials?title=[ja]	find all Tutorials which title contains 'ja'

example01: node, express and TypeORM project

- So the actual TypeORM syntax exists in entity / model and index.ts files.
- As advised during week 7 lectional, best place to learn TypeORM syntax is the official documentation page-
 - <https://typeorm.io/>
- The API methods are written in controller files
- The URL endpoints to access these are in routes files and
- You need to make index.ts aware of the routes file location

example01: node, express and TypeORM project

- You run the project as
 - npm run dev
 - You will see this

```
Server is running on port 8080.  
Executing (default): CREATE TABLE IF NOT EXISTS `tutorials` (`id` INTEGER NOT NU  
LL auto_increment , `title` VARCHAR(255), `description` VARCHAR(255), `published`  
TINYINT(1), `createdAt` DATETIME NOT NULL, `updatedAt` DATETIME NOT NULL, PRIM  
ARY KEY (`id`)) ENGINE=InnoDB;  
Executing (default): SHOW INDEX FROM `tutorials`
```

- As you can see TypeORM is generating SQL to create 'tutorial' table in the Cloud MS SQL database
- The project will run the API so keep it running till you launch the React app

example01: node, express and TypeORM project

- API will run as
 - <http://localhost:3001/api/tutorials>
 - Why?
 - Because you defined it so
 - Route was defined in `index.js`
 - API URL in `routes` file
- Each time API is called you will see SQL generated by TypeORM in the terminal/command prompt window where you are running the node, express.. project.

example01: react front end app

- Once the backend ie node, express and TypeORM is created and running- you can start working on the frontend app ie the react app
- You will expose the API created in node / express project to the react app
- React app will then make use of API methods to interact with the Cloud MS SQL database.
- After you have created a React app with create-next-app, you need to add requisite modules
 - `npm install axios`
 - Axios will help you consume API methods

example01: react front end app

- `components/`
 - the necessary components for the react app
- `services/`
 - create a service that uses axios object from `api.js` to send HTTP requests.

Database programming: multiple tables

- We have now seen two examples of full stack development
 1. Example 1 from week 7 lectional where we had a 'user's table in MS SQL database
 2. Example 1 from this week's lectional where we have a 'tutorials' table in MS SQL database
- In both of these examples we only dealt with 1 table per database- this is not always the case
- What if we had to deal with multiple tables
 - How do we enforce relationships between them?
 - How do we create joins between tables using TypeORM?

Multiple tables

- Time to look at an example with multiple tables
- So what if we had a 'tutorials' and a 'comments' table in the database.
- Let us go through an example of node, express and TypeORM example for this scenario
- Before we go ahead and do that, let us go back to TypeORM to learn how one deals with multiple tables?
 - We need to set up primary-foreign key relationship between them, if that is the case
 - We need to set up association between them ie . 1 to 1 (1:1), 1 to many (1:N) or many to many (M:N)
 - Then we need to write a query involving joins

Multiple tables

- Time to look at an example with multiple tables
 - example 2 
 - There is **no** react app to consume the API for this example- *it is left as a self-exercise*
- Model folder will have two files this time- one for tutorials table and one for comments table
- The important file to look at is the src/entity/Tutorial.ts-this is where we set up the relationship
- Look at the next slide

example02: Multiple tables

We set up

- Primary-Foreign key relationship
- 1-to-many relationship between tutorials and comments tables(s).

We use the `@ManyToOne` and `@OneToMany` decorators to define this relationship

example02: Multiple tables

- *Where is the actual join between tutorials and comments table?*
- Look at the controller file- where we write an API method – to get the comments for a particular tutorial
- Join using relations.
- We use a feature known as *eager loading!*

TypeORM *Eager Loading*

- *Eager Loading*
 - is the act of querying data of several models at once (one 'main' model and one or more associated models).
 - At the SQL level, this is a query with one or more joins).
- When this is done,
 - the associated models will be added by TypeORM in appropriately named, automatically created field(s) in the returned objects.
- In TypeORM, eager loading is done by specifying it in the Entity decorator as:
 - eager: true or false
 - lazy: true or false

Example on the Comment entity

```
@ManyToOne(() => Tutorial, (tutorial) => tutorial.comments, {  
    lazy: false, // Enable lazy loading for the tutorial relationship,  
    // which means the related tutorial will only be loaded when  
    accessed,  
    // improving performance by avoiding unnecessary data  
    retrieval.  
    eager: true, // Use eager loading to automatically load the  
    related tutorial whenever a comment is fetched,  
})
```

What about *many to many* relationship?

- That is not as complicated as it sounds
- Some tables in database have many to many (M:N) relationship
- You can enforce that via TypeORM
- But we do not want to cover everything today
- We will cover an example of *many to many* (M:N) in next week's lab.
- We will also make our front end interesting by displaying the data in a *paginated* manner



or



TypeORM documentation

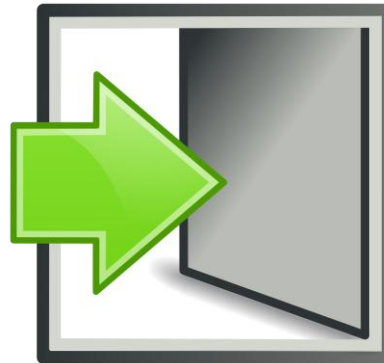
- Once again, if you want to learn and read more about multiple tables
- Read TypeORM official documentation
- <https://orkhan.gitbook.io/typeorm/docs/relations>

Segment 2

Authentication

Writing your own

Use 3rd party authentication using OAuth



This Photo by Unknown
Author is licensed under
CC BY-SA

Authentication

- Normally authentication is an essential part of many web applications
- You implemented registration and login-logout by storing user details in the earlier part of the course using localStorage
- Let us now make this more interesting!
- While implementing authentication, you can
 1. Write your custom logic, *like you did earlier with localStorage*, except use DATABASE for storing user data
 2. Use 3rd party authentication schemes such as – Google, Outlook, Facebook, etc- *not needed but you can try in your own free time*
 3. Using Auth0 service- outside the scope of this course

Authentication: custom logic

- Consider the scenario from assignment 2
- User registers and can then login
- The user details are stored in your Cloud MS SQL database
- As such all logic of registration and login is written by you
- So let us orchestrate the above scenario
 1. Node-Express-TypeORM project will provide you with an API to register, login user, make posts. The API will store user and corresponding post data into Cloud MS SQL

Authentication: custom logic

2. We also have to make sure that user password is stored in a hashed format in the database during registration process- *this will make our web application more secure*
3. React frontend app will talk to the API created above to implement the
 - User registration feature
 - User can login and
 - User can make a post which is saved into the database
- By now, you know everything that is needed to write code for this example except the *hashing part!*

Hashing user passwords

- What is meant by the term *hashing*?
- As defined by wiki- “.... *Hashing is the process of transforming any given key or a string of characters into another value. This is usually represented by a shorter, fixed-length value or key that represents and makes it easier to find or employ the original string.*”
- Various hashing algorithms exists such as:
 - Secure Hash Algorithm 1 (SHA-1)
 - Secure Hash Algorithm 2 (SHA-2)
 - Secure Hash Algorithm 3 (SHA-3)
 - MD2
 - MD4
 - MD5

Why is hashing used?

- For any reason, your database may be compromised and its data may be obtained by someone else.
 - If the passwords are in what we call plain text, you will have leaked a piece of sensitive information that your users have trusted you with!
- Instead of plain text, passwords are typically hashed with an *one-way* hashing function.
 - If the cryptographic function used for the hashing is strong, then the passwords are much safer, because even if someone gets their hands on your database, it's computationally infeasible to calculate the passwords.

Is hashing password enough?

- For commercial applications- NO!!
- Developers often use- *hashing and salting* together.
 - each password is hashed together with a unique randomly generated piece of input (called *salt*). The salt is stored in plain text in the database and it doesn't have to be secret, because its main purpose is to render precomputed hash dictionaries useless.
 - Almost all serious platforms use hashing and salting to store their users' passwords. Incidentally, this is exactly why you can reset your password in various services but almost never recover it: The hash can be overwritten by the system, but it can't be decrypted.

Hashing algorithm

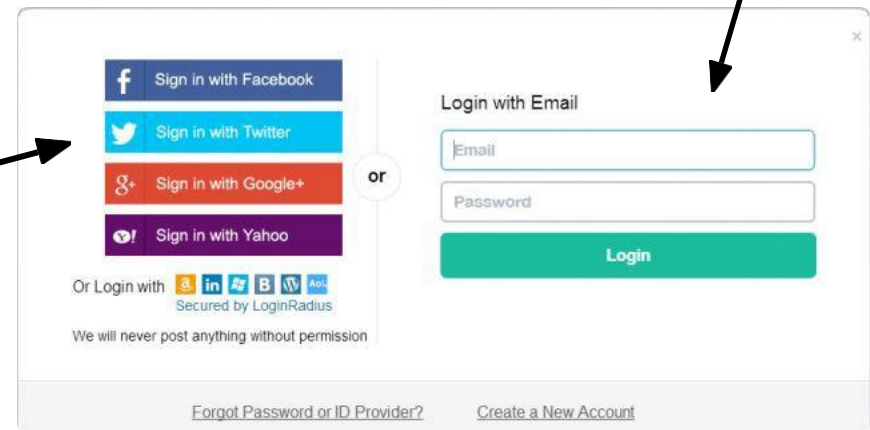
- You should not rely on MD5
- It is not considered cryptographically secure anymore- it has been broken down since mid 2000's.
- There are better algorithms available
- Since our logic of registration will be written in Node, Express project, we need a robust Node library to deal with hashing
- We will use a module known as
 - Argon2
 - Official documentation:
 - <https://www.ory.sh/choose-recommended-argon2-parameters-password-hashing/>

Argon2

- Argon2 is a cryptographic hash algorithm specifically designed to secure passwords.
- It is recommended by OWASP as a modern, secure and flexible algorithm.
- *Do you know about OWASP?*
- As aspiring web developers, you must!!
- OWASP (<https://owasp.org/>)
 - The Open Web Application Security Project® (OWASP) is a nonprofit foundation that works to improve the security of software. Through community-led open-source software projects, hundreds of local chapters worldwide, tens of thousands of members, and leading educational and training conferences, the OWASP Foundation is the source for developers and technologists to secure the web.

What if you do not want to write your authentication logic?

- Sometimes you do not want to write your authentication logic
- In that case, you can rely upon using 3rd party auth schemes such as
 - Google login
 - Outlook
 - Facebook
 - Twitter
 - *etc.*



This Photo by Unknown Author is licensed under CC BY-SA

- Obviously there will be a compromise!!
 - *Can you guess?*

Implementing third-party logins

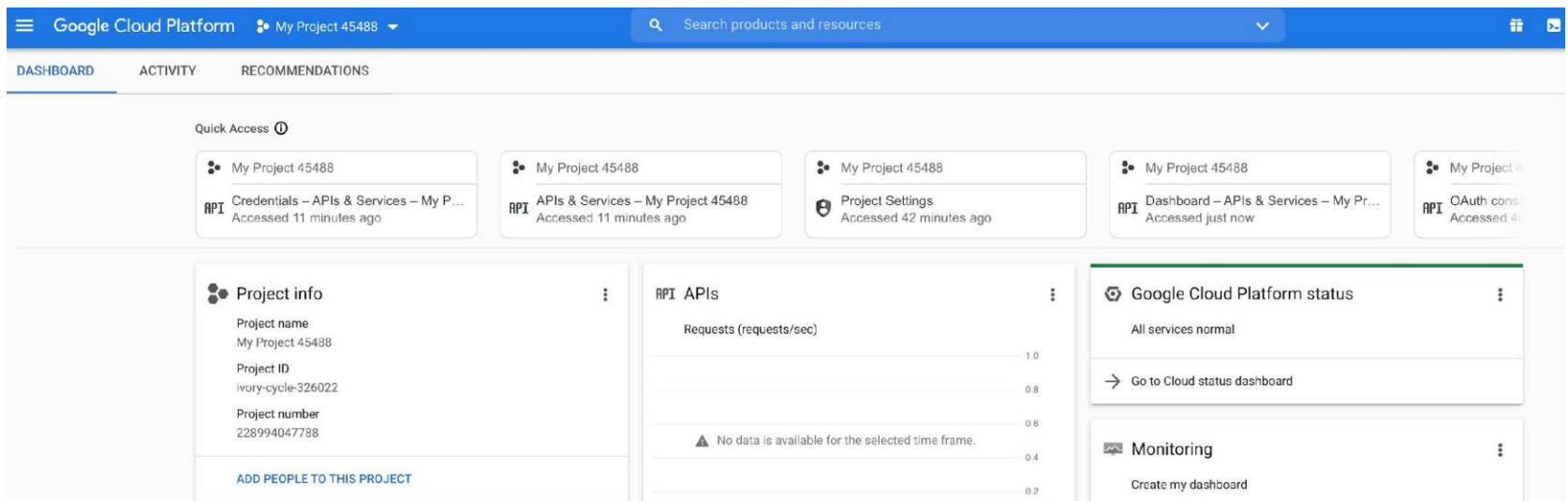
- ADVANTAGES
 - Do not have to worry about security
 - No need to write your own custom logic
 - Third party providers can generate analytics for your website, if requested
- DISADVANTAGES
 - Are not as easy as it appear
 - You have to learn
 - the auth APIs of the provider and
 - how to play with the provider's Cloud Platform Dashboards
 - In any case, it will make for some complicated and cumbersome code

Implementing GOOGLE login in a React app

- There are lot of good tutorials where you can find all the instructions
- Have a look at this one:
 - <https://blog.logrocket.com/guide-adding-google-login-react-app/>
- Follow all the instructions and see if you can implement a Google login

Implementing GOOGLE login in a React app

- Step 1: Create and configure project via Google Cloud Platform dashboard
 - <https://console.cloud.google.com>
- *NOTE: login via personal google account ie DO NOT use RMIT google account*



I want to learn more about Google Cloud Console

- Go to RMIT LinkedIn learning *login* page
 - <https://www.rmit.edu.au/library/borrowing-and-collections/collections/databases/linkedin-learning>
- Log in to access LinkedIn learning page
- Search for this video tutorial
 - Google Cloud Platform (GCP) Essential Training for Developers - by *Albert Padin*

Should I write my own logic or use these third party providers?

- It depends upon the project and its needs, requirements
- *Most React projects rely on custom authentication*
- *Some will rely on social media/3rd party login schemes*
- Security vulnerabilities may crop up with the 3rd party auth schemes
 - Social identity providers like Facebook and LinkedIn have faced infamous data breaches over the years, where leaks compromised millions of user accounts at a time.
 - Users who repeatedly use the same passwords will likely have multiple compromised accounts on their hands. Frequent social login users are those most at risk, as credential leaks jeopardise every app or site login linked to a breached social media account.
 - *And what these organisations accessing your private data?*

Commercial project and custom auth logic

- For our course, we are saving user data in a Cloud MS SQL database
- Not all commercial projects go that way
- The *enthusiasts* who have real interest in learning more to prepare for future life and work MUST
 - Learn how to implement authentication in React app using Auth0
 - Auth0 allows you to authenticate and authorize apps and APIs with any identity provider running on any stack on any device or cloud.
 - Here is their official page to implement authentication in a React app
 - <https://auth0.com/blog/complete-guide-to-react-user-authentication/>

References

- Reference: *The Road to React (2025 edition)*, by Robin Weiruch; Leanpub
- [<https://sequelize.org/>]
- [<https://www.bezkoder.com/react-node-express-mysql/>]
- [<https://www.veracode.com/blog/secure-development/zero-hashing-under-10-minutes-argon2-nodejs>]
- [<https://owasp.org/>]

Assignment 2 has been online

- Online!
- Full stack web application
- To be completed in a group
- Deadline: Sun 07.06.2025 (11:59 pm Melbourne time)
- Worth: 45%
- Requires use of
 - React, (Node, Express) & Cloud MS SQL database
 - GitHub on a regular basis

Next week

- More FSD
 - Migrations in TypeORM
 - GraphQL- *part 1*
 - *Unit testing*